Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования «Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)» (МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ: Инженерный бизнес и менеджмент
КАФЕДРА: ИБМ3 «Промышленная логистика»

ЛАБОРАТОРНАЯ РАБОТА

ПО ДИСЦИПЛИНЕ: “ПАРАДИГМЫ И КОНСТРУКЦИИ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ”

НА ТЕМУ: Лабораторная работа №2.

Студент: ИБМ3-34Б (Группа) **Сучкова Юлия Павловна** (И.О.Фамилия)

Преподаватель: _____ (И.О.Фамилия)

2026 г.

Условие задачи

Вариант 22

1. Необходимо создать виртуальное окружение и установить в него хотя бы один внешний пакет с использованием pip.

2. Необходимо разработать программу, реализующую работу с классами. Программа должна быть разработана в виде консольного приложения на языке Python 3.

3. Все файлы проекта (кроме основного файла main.py) должны располагаться в пакете lab_python_oop.

4. Каждый из нижеперечисленных классов должен располагаться в отдельном файле пакета lab_python_oop.

Задание по варианту (22). Написать программу, в которой описана иерархия классов: товар (игрушка, продукт, молочный продукт). Базовый класс должен иметь метод вывода на экран характеристик экземпляра.

Текст программы

Код базового класса tovar.py:

```
class Tovar:

    def __init__(self, name, price):
        self.name = name
        self.price = price

    def get_type_name(self):
        return "Товар"

    def display_info(self):
        print(f"=== {self.get_type_name()} ===")
        print(f"Название: {self.name}")
        print(f"Цена: {self.price:.2f} руб.")

    def get_info(self):
        return {
            "Тип": self.get_type_name(),
            "Название": self.name,
            "Цена": f"{self.price:.2f}"
        }
```

Коды наследуемых классов igrushka.py, product.py:

```
from .tovar import Tovar

class Igrushka(Tovar):
    def __init__(self, name, price, material, min_age):
        super().__init__(name, price)
        self.material = material
        self.min_age = min_age

    def get_type_name(self):
        return "Игрушка"

    def display_info(self):
        super().display_info()
        print(f"Материал: {self.material}")
        print(f"Минимальный возраст: {self.min_age}+")

    def get_info(self):
        info = super().get_info()
        info.update({
            "Материал": self.material,
            "Мин. возраст": f"{self.min_age}+"
        })
        return info


from .tovar import Tovar

class Produkt(Tovar):
    def __init__(self, name, price, expiration_date, weight):
        super().__init__(name, price)
        self.expiration_date = expiration_date
        self.weight = weight

    def get_type_name(self):
        return "Продукт"

    def display_info(self):
        super().display_info()
        print(f"Срок годности: {self.expiration_date}")
```

```
        print(f"Вес: {self.weight} г")

    def get_info(self):
        info = super().get_info()
        info.update({
            "Срок годности": self.expiration_date,
            "Вес": f"{self.weight} г"
        })
        return info
```

Код класса, наследующего product – molochny_product.py:

```
from .produkt import Produkt

class MolochnyProdukt(Produkt):
    def __init__(self, name, price, expiration_date, weight, fat_content, storage_temp):
        super().__init__(name, price, expiration_date, weight)
        self.fat_content = fat_content
        self.storage_temp = storage_temp

    def get_type_name(self):
        return "Молочный продукт"

    def display_info(self):
        super().display_info()
        print(f"Жирность: {self.fat_content}%")
        print(f"Температура хранения: {self.storage_temp} °C")

    def get_info(self):
        info = super().get_info()
        info.update({
            "Жирность": f"{self.fat_content}%",
            "Темп. хранения": f"{self.storage_temp} °C"
        })
        return info
```

Код для init:

```
from .tovar import Tovar
from .igrushka import Igrushka
from .produkt import Produkt
from .molochny_produkt import MolochnyProdukt

__all__ = ["Tovar", "Igrushka", "Produkt", "MolochnyProdukt"]
```

Код main.py:

```
from lab_python_oop import Tovar, Igrushka, Produkt, MolochnyProdukt
from tabulate import tabulate

def main():
    goods = [
        Igrushka("Кукла Маша", 599.99, "пластик", 3),
        Produkt("Хлеб", 45.50, "2026-10-12", 500),
        MolochnyProdukt("Молоко", 89.90, "2026-10-15", 1000, 3.2, 4),
        Tovar("Пакет", 5.00),
    ]

    print("Характеристики экземпляров:\n")
    for item in goods:
        item.display_info()
        print()

    print("Сводная таблица:")
    headers = [
        "Тип", "Название", "Цена",
```

```
            "Материал", "Мин. возраст",
            "Срок годности", "Вес",
            "Жирность", "Темп. хранения"
        ]
        rows = []
        for item in goods:
            info = item.get_info()
            rows.append([info.get(h, "") for h in headers])

        print(tabulate(rows, headers=headers, tablefmt="grid"))

if __name__ == "__main__":
    main()
```

Тестовый запуск кода

```
Характеристики экземпляров:

=== Игрушка ===
Название: Кукла Маша
Цена: 599.99 руб.
Материал: пластик
Минимальный возраст: 3+

=== Продукт ===
Название: Хлеб
Цена: 45.50 руб.
Срок годности: 2026-10-12
Вес: 500 г

=== Молочный продукт ===
Название: Молоко
Цена: 89.90 руб.
Срок годности: 2026-10-15
Вес: 1000 г
Жирность: 3.2%
Температура хранения: 4 °C

=== Товар ===
Название: Пакет
Цена: 5.00 руб.
```

```
Сводная таблица:
+------------------+------------+--------+------------+---------------+-----------------+--------+------------+------------------+
| Тип              | Название   |   Цена | Материал   | Мин. возраст  | Срок годности   | Вес    | Жирность   | Темп. хранения   |
+==================+============+========+============+===============+=================+========+============+==================+
| Игрушка          | Кукла Маша | 599.99 | пластик    | 3+            |                 |        |            |                  |
+------------------+------------+--------+------------+---------------+-----------------+--------+------------+------------------+
| Продукт          | Хлеб       |   45.5 |            |               | 2026-10-12      | 500 г  |            |                  |
+------------------+------------+--------+------------+---------------+-----------------+--------+------------+------------------+
| Молочный продукт | Молоко     |   89.9 |            |               | 2026-10-15      | 1000 г | 3.2%       | 4 °C             |
+------------------+------------+--------+------------+---------------+-----------------+--------+------------+------------------+
| Товар            | Пакет      |      5 |            |               |                 |        |            |                  |
+------------------+------------+--------+------------+---------------+-----------------+--------+------------+------------------+
```

Дополнительное задание

Дополнительное задание

Протестируйте корректность работы Вашей программы с помощью **модульного теста**.

Код для доп. задания:

```
import unittest
from lab_python_oop import Tovar, Igrushka, Produkt, MolochnyProdukt

class TestGoods(unittest.TestCase):

    def test_tovar_creation(self):
        t = Tovar("Пакет", 5.0)
        self.assertEqual(t.name, "Пакет")
        self.assertEqual(t.price, 5.0)
        self.assertEqual(t.get_type_name(), "Товар")

    def test_igrushka_creation(self):
        i = Igrushka("Кукла", 100.0, "пластик", 3)
        self.assertEqual(i.name, "Кукла")
        self.assertEqual(i.material, "пластик")
        self.assertEqual(i.get_type_name(), "Игрушка")

    def test_molochny_produkt_creation(self):
        m = MolochnyProdukt("Молоко", 80.0, "2026-10-15", 1000, 3.2, 4)
        self.assertEqual(m.name, "Молоко")
        self.assertEqual(m.expiration_date, "2026-10-15") # Из Продукта
        self.assertEqual(m.fat_content, 3.2)
        self.assertEqual(m.get_type_name(), "Молочный продукт")

    def test_get_info_dict(self):
        p = Produkt("Хлеб", 40.0, "2026-10-12", 500)
        info = p.get_info()

        self.assertIn("Тип", info)
        self.assertIn("Название", info)
        self.assertIn("Вес", info)
        self.assertEqual(info["Тип"], "Продукт")

if __name__ == '__main__':
    unittest.main()
```

Вывод программы:

```
(.venv) PS C:\Users\okjul\OneDrive\Desktop\Лабы по парадигмам (питон)\ЛР2\lab2> python test_lab.py
....
----------------------------------------------------------------------
Ran 4 tests in 0.001s
```